SLIIT

COMPUTING | BUSINESS | ENGINEERING

IE3090
Network Programming
3rd Year, Semester I

Assignment Report

RemoteOps – Assignment Part 1

A Remote System Monitoring and Management Tool over TCP/IP

Submitted to

Sri Lanka Institute of Information Technology

In partial fulfillment of the requirements for the

Bachelor of Science Special Honors Degree in Information Technology

5th October 2026

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text.

Registration Number : IT24101899

Name : Kavirathna N R

Table of Contents

1. Registration Number and Personalised Values 1
2. Architecture Overview 2
2.1 Components 2
2.2 Diagram 2
2.3 Concurrency model: one thread per connection 3
2.4 Shared state and thread safety 4
3. Protocol Implementation 5
3.1 Framing 6
3.2 Protocol evidence (full request/response exchange) 7
4. Personalisation Proof 8
4.1 Agent listening on port 9410 8
4.2 Personalised log file 8
4.3 Personalised storage path 8
5. Annotated Code 9
5.1 Personalised constants and session structure 9
5.2 Connection handling and concurrency 10
5.3 Line framing and byte counting 11
5.4 Authentication and command dispatch 12
5.5 SYSINFO and LISTPROC 13
5.6 Restricted EXEC 14
5.7 File upload and download 15
5.8 UDP monitoring loop 16
5.9 Logging 17
5.10 Controller 18
6. Execution and Output Screenshots 19
6.1 Agent starting up 21
6.2 Successful AUTH 21
6.3 SYSINFO, LISTPROC and EXEC 22
6.4 Successful PUT and GET 22
6.5 UDP monitoring stream 23
6.6 Error cases handled gracefully 23
6.7 Five simultaneous Controllers 24
7. Testing Summary 25
8. Design Rationale and Assumptions 27
8.1 Key design decisions 27
8.2 Assumptions where the brief leaves a choice 28
9. Optional Extension and Limitations 29
10. Build and Run Instructions 30

List of Figures

Figure 2.1: RemoteOps architecture (Controller, Agent, threads and channels)
Figure 3.1: Full protocol session showing the SID tag
Figure 4.1: Agent listening on port 9410
Figure 4.2: Contents of remoteops_IT24101899.log
Figure 4.3: Uploaded file in ./agentfiles/IT24101899
Figure 5.1: Constants and session_t structure (agent_899.c)
Figure 5.2: main() accept loop and client_thread()
Figure 5.3: read_line() and recv_bytes()
Figure 5.4: handle_line() authentication and dispatch
Figure 5.5: get_sysinfo() and do_listproc()
Figure 5.6: WHITELIST table and do_exec()
Figure 5.7: valid_filename(), reject_put(), do_put() and do_get()
Figure 5.8: monitor_thread(), start_monitor() and stop_monitor()
Figure 5.9: logmsg()
Figure 5.10: cmd_put(), cmd_get(), cmd_monitor() and udp_thread() (controller_899.c)
Figure 6.1: Agent starting up
Figure 6.2: Successful AUTH
Figure 6.3: SYSINFO, LISTPROC and EXEC
Figure 6.4: Successful PUT and GET
Figure 6.5: UDP monitoring stream
Figure 6.6: Error cases handled gracefully
Figure 6.7: Five simultaneous Controllers0

1. Registration Number and Personalised Values

Registration number: **IT24101899**. Numeric part: **24101899**. Every value below is calculated from it using the formulas in section 2.4 of the assignment brief, and the same values appear in the README.

Item	Calculation	Value
Agent listening port	7000 + first four digits of 24101899 (2410)	**9410**
Source files	last three digits = 899	`agent_899.c`, `controller_899.c`, `Makefile_899`
Session ID (SID) tag	last four digits 1899, reversed	**SID:9981**
Authentication token	"OPS-" + last four digits (1899)	**OPS-1899**
Log file name	remoteops_<full registration number>.log	`remoteops_IT24101899.log`
File storage path	./agentfiles/<full registration number>/<filename>	`./agentfiles/IT24101899/<filename>`
Submission archive	IE3090_<full registration number>.zip	`IE3090_IT24101899.zip`

In the source code these appear as named constants at the top of `agent_899.c` (`AGENT_PORT 9410`, `SID_TAG "9981"`, `TOKEN "OPS-1899"`, `LOG_FILE`, `STORE_DIR`) and in `controller_899.c`, rather than as numbers scattered through the code, so each value is defined in a single place.

2. Architecture Overview

2.1 Components

- **Agent** (`agent_899`) – the server on the managed machine. It listens on TCP port 9410, authenticates each Controller, executes protocol commands, stores files, logs events and sends UDP monitoring datagrams.
- **Controller** (`controller_899`) – the administrator's interactive client. It sends protocol commands, performs PUT/GET file transfers and receives the UDP monitoring stream on a local UDP port.
- **Two channels** – a TCP control channel for all commands and file transfers, and a UDP channel used only for the periodic monitoring datagrams.

2.2 Diagram

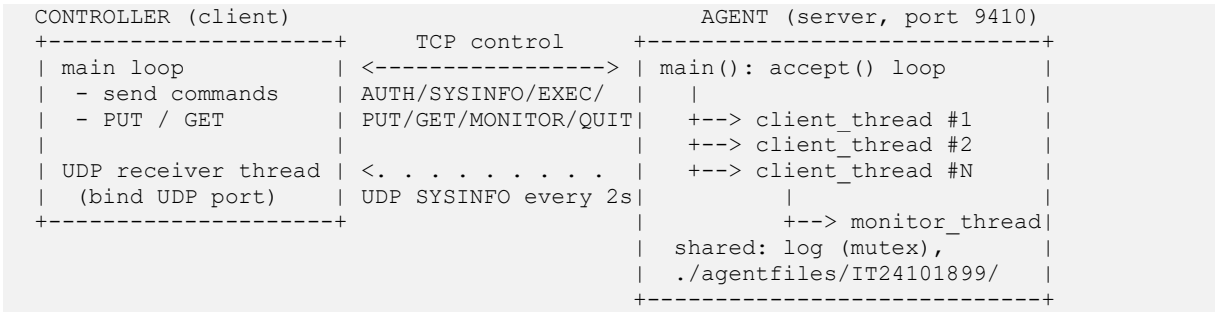


Figure 2.1: RemoteOps architecture (Controller, Agent, threads and channels)

2.3 Concurrency model: one thread per connection

The main thread only calls `accept()`. For each new connection it allocates a `session_t` with `calloc()` (socket, peer address, authentication flag, receive buffer, monitoring state) and starts a detached POSIX thread running `client_thread()`. Each thread handles exactly one Controller using ordinary blocking I/O. While a session is monitoring, it owns one extra helper thread (`monitor_thread`) that sends the UDP datagrams.

Why threads rather than the alternatives:

- **Versus fork():** forked processes have separate memory, so the shared log file and per-session state would need inter-process locking or IPC. Threads share the address space and need just one mutex around the log.
- **Versus select/poll/epoll:** a single-threaded event loop would need an explicit state machine per client for partial lines and half-finished file transfers. Blocking calls such as `popen()` (LISTPROC, EXEC) and large PUT/GET transfers would also stall every other client.
- **Result:** each handler is simple sequential code, a slow client only blocks its own thread, and disconnects are handled in one place (the end of `client_thread`). It meets the requirement of at least 5 simultaneous Controllers (tested with 6).

Trade-off: thread-per-connection does not scale to thousands of clients, and the Agent sets no upper limit on the number of threads (`MAX_CLIENTS` is only the `listen()` backlog). This is acceptable for a management tool with a handful of administrators and is listed as a limitation in section 9.

2.4 Shared state and thread safety

- The log file is the only globally shared resource; `logmsg()` protects it with a mutex (`g_log_mutex`) and flushes each timestamped line.
- All other state lives in the per-connection `session_t`, so client threads share nothing else.
- The monitor thread is stopped with the `mon_stop` flag, which is read and written under the session's `mon_mutex`, and is then joined with `pthread_join()`, so no stream outlives its session.
- `SIGPIPE` is ignored and `send()` uses `MSG_NOSIGNAL`, so writing to a vanished client cannot crash the Agent. `strtok_r()` and `localtime_r()` are used because they are thread-safe.

3. Protocol Implementation

The fixed line-based protocol from section 2.3 of the brief is implemented exactly as specified. No message formats were invented. Every command and response is one line ending in `\n`, except the raw bytes that follow PUT and GET.

Command	Success response	Error responses	Handler
`AUTH <token>`	`OK AUTHENTICATED SID:9981`	`ERR 001 AUTH_FAILED`	`handle_line`
`SYSINFO`	`OK SYSINFO <cpu> <mem_mb> <uptime> SID:9981`	`ERR 003 NOT_AUTHENTICATED`	`do_sysinfo`
`LISTPROC`	`OK PROCS <pid:name,...> SID:9981`	`ERR 011 INTERNAL_ERROR`	`do_listproc`
`EXEC <name>`	`OK EXEC_RESULT <output> SID:9981`	`ERR 002 COMMAND_NOT_ALLOWED`	`do_exec`
`PUT <name> <size>` + bytes	`OK FILE_RECEIVED <name> SID:9981`	`ERR 004 FILE_TOO_LARGE`, `ERR 006 INVALID_FILENAME`, `ERR 008 BAD_REQUEST`	`do_put`
`GET <name>`	`OK FILE_SEND <name> <size> SID:9981` + bytes	`ERR 005 FILE_NOT_FOUND`	`do_get`
`MONITOR START <port>`	`OK MONITOR_STARTED SID:9981`	`ERR 009 ALREADY_MONITORING`, `ERR 008 BAD_REQUEST`	`start_monitor`
`MONITOR STOP`	`OK MONITOR_STOPPED SID:9981`	`ERR 010 NOT_MONITORING`	`stop_monitor`
`QUIT`	`OK BYE SID:9981`	_	`handle_line`

Other error codes: `ERR 007 UNKNOWN_COMMAND` for an unrecognised command and `ERR 003 NOT_AUTHENTICATED` for any command other than AUTH sent before successful authentication. Reason codes are self-explanatory and used consistently.

3.1 Framing

- **Partial lines:** each session keeps a receive buffer (`rxbuf`). `read_line()` searches it for `\n` and calls `recv()` again if the line is incomplete, so a command split over several `recv()` calls is reassembled.
- **Several lines in one buffer:** bytes after the first `\n` stay in `rxbuf` and are used by the next `read_line()` call without another `recv()`.
- **PUT/GET byte counting:** `recv_bytes()` first uses any bytes already in `rxbuf` after the PUT line, then keeps calling `recv()` until exactly `<filesize>` bytes have been read. GET sends exactly `st_size` bytes using a `send()` loop. Lines and file data share the same buffer, so the stream stays correctly framed.
- **Rejected uploads:** when a PUT is rejected (too large or bad filename) the Agent still reads and discards the announced bytes (`reject_put`) before sending the error, so the payload is never misread as commands.
- **Robust line handling:** a trailing `\r` is removed, so CRLF line endings are accepted, and a blank line is ignored instead of being treated as a disconnect.
- **SID tag:** the function `reply()` appends `SID:9981` to every OK and ERR line in one place, so no response can omit it. The UDP datagrams carry the same tag.

3.2 Protocol evidence (full request/response exchange)

```
Nethma@192:~/IE3090_RemoteOps_IT24101899$ cd ~/IE3090_RemoteOps_IT24101899
Nethma@192:~/IE3090_RemoteOps_IT24101899$ ./controller_899 127.0.0.1
Connected to Agent 127.0.0.1:9410 (SID expected SID:9981)

Commands:
  auth <token>          e.g. auth OPS-1899
  sysinfo
  listproc
  exec <name>           DATE | UPTIME | DISKFREE | HOSTNAME | WHOAMI
  put <local> <remote>
  get <remote> <local>
  monitor start         (prompts for UDP port)
  monitor stop
  quit
  help

controller> auth OPS-1899
<< OK AUTHENTICATED SID:9981
controller> sysinfo
<< OK SYSINFO 0.16 1384 9790 SID:9981
controller> listproc
<< OK PROCs ______1_systemd,______2_kthreadd,______3_pool_workqueue_release,
______4_kworker/R-rcu_gp,______5_kworker/R-sync_wq,______6_kworker/R-kvfree_
rcu_reclaim,______7_kworker/R-slub_flushwq,______8_kworker/R-netns,_____12_k
worker/u16:0-events_unbound,_____13_kworker/u16:1-ipv6_addrconf,_____14_kwor
ker/R-mm_percpu_wq,_____15_ksoftirqd/0,_____16_rcu_preempt,_____17_rcu_exp_p
ar_gp_kthread_worker/0,_____18_rcu_exp_gp_kthread_worker,_____19_migration/0
,_____20_idle_inject/0,_____21_cpuhp/0,_____22_cpuhp/1,_____23_idle_inject/1
,_____24_migration/1,_____25_ksoftirqd/1,_____28_cpuhp/2,_____29_idle_inject
/2,_____30_migration/2,_____31_ksoftirqd/2,_____33_kworker/2:0H-events_highp
ri,_____34_cpuhp/3,_____35_idle_inject/3,_____36_migration/3,_____37_ksoftir
qd/3,_____47_kdevtmpfs,_____48_kworker/R-inet_frag_wq,_____49_rcu_tasks_kthr
ead,_____50_rcu_tasks_rude_kthread,_____51_rcu_tasks_trace_kthread,_____52_k
worker/u20:1-kvfree_rcu_reclaim,_____53_kauditd,_____54_khungtaskd,_____55_o
om_reaper SID:9981
controller> exec DATE
<< OK EXEC_RESULT Sun Oct  4 01:55:03 PM EDT 2026 SID:9981
controller> exec HOSTNAME
<< OK EXEC_RESULT 192.168.1.6 SID:9981
controller> put testfile.txt testfile.txt
[throughput] PUT 1019368 bytes/s
<< OK FILE_RECEIVED testfile.txt SID:9981
controller> get testfile.txt roundtrip.txt
```

```
controller> get testfile.txt roundtrip.txt
<< OK FILE_SEND testfile.txt 37 SID:9981
[throughput] GET 77619 bytes/s
Saved roundtrip.txt (37 bytes)
controller> quit
<< OK BYE SID:9981
Bye.
Nethma@192:~/IE3090_RemoteOps_IT24101899$
```

Figure 3.1: Full protocol session showing the SID tag

4. Personalisation Proof

The three figures below show that the Agent really uses the personalised values.

4.1 Agent listening on port 9410

```
Last login: Sun Oct  4 12:09:21 2026 from 192.168.56.1
Nethma@192:~$ cd ~
Nethma@192:~$ cd IE3090_RemoteOps_IT24101899
Nethma@192:~/IE3090_RemoteOps_IT24101899$ nano controller_899.c
Nethma@192:~/IE3090_RemoteOps_IT24101899$ cd ~/IE3090_RemoteOps_IT24101899
Nethma@192:~/IE3090_RemoteOps_IT24101899$ ss -tlnp | grep 9410
LISTEN 0      64           0.0.0.0:9410      0.0.0.0:*    users:(("agent_899
",pid=35708,fd=4))
Nethma@192:~/IE3090_RemoteOps_IT24101899$
```

Figure 4.1: Agent listening on port 9410

4.2 Personalised log file

```
Nethma@192:~/IE3090_RemoteOps_IT24101899$ head -20 remoteops_IT24101899.log
[2026-10-04 12:50:56] === RemoteOps Agent starting port=9410 SID=9981 ===
[2026-10-04 12:50:56] Listening on port 9410
[2026-10-04 12:54:21] CONNECTED 127.0.0.1:51624
[2026-10-04 12:54:42] CMD [127.0.0.1:51624] AUTH OPS-1899
[2026-10-04 12:54:42] AUTH OK from 127.0.0.1:51624
[2026-10-04 12:56:08] CMD [127.0.0.1:51624] SYSINFO
[2026-10-04 12:56:08] SYSINFO -> cpu=0.02 mem=1381 uptime=6266
[2026-10-04 12:56:13] CMD [127.0.0.1:51624] LISTPROC
[2026-10-04 12:56:13] LISTPROC -> ______1_systemd,______2_kthreadd,______3_pool_workqueue_release,______4_kworker/R-rcu_gp,______5_kworker/R-sync_wq,______6
_kworker/R-kvfree_rcu_reclaim,______7_kworker/R-slub_flushwq,______8_kworker/R-netns,______9_kworker/0:0-ata_sff,_____12_kworker/u16:0-events_unbound,_____1
3_kworker/u16:1-ipv6_addrconf,_____14_kworker/R-mm_percpu_wq,_____15_ksoftirqd/0,_____16_rcu_preempt,_____17_rcu_exp_par_gp_kthread_worker/0,_____18_rcu_exp
_gp_kthread_worker,_____19_migration/0,_____20_idle_inject/0,_____21_cpuhp/0,_____22_cpuhp/1,_____23_idle_inject/1,_____24_migration/1,_____25_ksoftirqd/1,_
____28_cpuhp/2,_____29_idle_inject/2,_____30_migration/2,_____31_ksoftirqd/2,_____33_kworker/2:0H-events_highpri,_____34_cpuhp/3,_____35_idle_inject/3,_____
36_migration/3,_____37_ksoftirqd/3,_____40_kworker/u17:0-events_unbound,_____45_kworker/u19:1-events_unbound,_____47_kdevtmpfs,_____48_kworker/R-inet_frag_w
q,_____49_rcu_tasks_kthread,_____50_rcu_tasks_rude_kthread,_____51_rcu_tasks_trace_kthread,_____52_kworker/u20:1-flush-253:0
[2026-10-04 12:56:21] CMD [127.0.0.1:51624] EXEC DATE
[2026-10-04 12:56:21] EXEC DATE -> Sun Oct  4 12:56:21 PM EDT 2026
[2026-10-04 12:56:42] CMD [127.0.0.1:51624] EXEC HOSTNAME
[2026-10-04 12:56:42] EXEC HOSTNAME -> 192.168.1.6
[2026-10-04 12:57:48] CMD [127.0.0.1:51624] PUT testfile.txt 37
[2026-10-04 12:57:48] PUT testfile.txt (37 bytes) from 127.0.0.1:51624
[2026-10-04 12:58:22] CMD [127.0.0.1:51624] GET testfile.txt
[2026-10-04 12:58:22] GET testfile.txt (37 bytes) to 127.0.0.1:51624
Nethma@192:~/IE3090_RemoteOps_IT24101899$
```

Figure 4.2: Contents of remoteops_IT24101899.log

4.3 Personalised storage path

```
Nethma@192:~/IE3090_RemoteOps_IT24101899$ cd ~/IE3090_RemoteOps_IT24101899
Nethma@192:~/IE3090_RemoteOps_IT24101899$ ls -l agentfiles/IT24101899/
total 4
-rw-r--r--. 1 Nethma Nethma 37 Oct  4 12:57 testfile.txt
Nethma@192:~/IE3090_RemoteOps_IT24101899$
```

Figure 4.3: Uploaded file in ./agentfiles/IT24101899/

5. Annotated Code

Each subsection explains what a key part of the source code does and is followed by a screenshot of that code.

5.1 Personalised constants and session structure

The `#define` block holds every value derived from the registration number (port, SID tag, interval). `session_t` holds everything one connection needs: the socket, peer address and port, an `authed` flag, the receive buffer used for framing, and the monitoring state (`mon_active`, `mon_stop`, `mon_mutex`, UDP socket and the Controller's UDP address). Because all state is per-session, client threads share nothing except the log.

```
/* ===== PERSONALISED CONSTANTS ===== */
#define AGENT_PORT       9410
#define SID_TAG          "9981"
#define TOKEN            "OPS-1899"
#define LOG_FILE         "remoteops_IT24101899.log"
#define STORE_DIR        "./agentfiles/IT24101899"
#define MAX_UPLOAD_MB    10
#define MAX_UPLOAD_BYTES (MAX_UPLOAD_MB * 1024 * 1024)
#define MON_INTERVAL_SEC 2
#define BUF_SIZE         4096
#define MAX_CLIENTS      64
```

Figure 5.1a: Personalised constants (agent_899.c)

```
/* ===== SESSION ===== */
typedef struct {
    int    fd;
    char   peer[INET_ADDRSTRLEN];
    int    peer_port;
    int    authed;
    int    mon_active;
    int    mon_stop;
    int    udp_fd;
    struct sockaddr_in ctrl_udp;
    pthread_t mon_tid;
    pthread_mutex_t mon_mutex;
    char   rxbuf[BUF_SIZE];
    int    rxlen;
} session_t;
```

Figure 5.1b: session_t structure (agent_899.c)

5.2 Connection handling and concurrency

`main()` creates the storage folders, opens the log, ignores `SIGPIPE`, creates the listening socket (with `SO_REUSEADDR`), binds to port 9410 and calls `listen()`. It then loops on `accept()`; for every connection it allocates a `session_t`, records the peer address with `inet_ntop()`, initialises `mon_mutex` and starts a detached thread. `client_thread()` reads lines until the client quits or disconnects, logging each command and passing it to `handle_line()`. At the end it stops any running monitor, closes the sockets, logs DISCONNECTED and frees the session.

```
/* ===== MAIN ===== */
int main(void)
{
    mkdir("./agentfiles", 0755);
    mkdir(STORE_DIR, 0755);

    g_logfp = fopen(LOG_FILE, "a");
    if (!g_logfp) { perror("log"); return 1; }
    logmsg("=== RemoteOps Agent starting port=%d SID=%s ===", AGENT_PORT, SID_TAG);

    signal(SIGPIPE, SIG_IGN);

    int lfd = socket(AF_INET, SOCK_STREAM, 0);
    if (lfd < 0) { perror("socket"); return 1; }

    int opt = 1;
    setsockopt(lfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in addr = {0};
    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port        = htons(AGENT_PORT);

    if (bind(lfd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind"); close(lfd); return 1;
    }
    if (listen(lfd, MAX_CLIENTS) < 0) { perror("listen"); return 1; }

    printf("RemoteOps Agent listening on port %d (SID %s)\n", AGENT_PORT, SID_TAG);
    fflush(stdout);
    logmsg("Listening on port %d", AGENT_PORT);

    while (1) {
        struct sockaddr_in ca;
        socklen_t cl = sizeof(ca);
        int cfd = accept(lfd, (struct sockaddr *)&ca, &cl);
        if (cfd < 0) { if (errno == EINTR) continue; perror("accept"); continue; }

        session_t *s = calloc(1, sizeof(session_t));
        if (!s) { close(cfd); continue; }
        s->fd = cfd;
        inet_ntop(AF_INET, &ca.sin_addr, s->peer, sizeof(s->peer));
:
```

```
    while (1) {
        struct sockaddr_in ca;
        socklen_t cl = sizeof(ca);
        int cfd = accept(lfd, (struct sockaddr *)&ca, &cl);
        if (cfd < 0) { if (errno == EINTR) continue; perror("accept"); continue; }

        session_t *s = calloc(1, sizeof(session_t));
        if (!s) { close(cfd); continue; }
        s->fd = cfd;
        inet_ntop(AF_INET, &ca.sin_addr, s->peer, sizeof(s->peer));
        s->peer_port = ntohs(ca.sin_port);
        pthread_mutex_init(&s->mon_mutex, NULL);

        pthread_t tid;
        if (pthread_create(&tid, NULL, client_thread, s) != 0) {
            perror("pthread_create"); close(cfd); free(s); continue;
        }
        pthread_detach(tid);
    }

    close(lfd);
    if (g_logfp) fclose(g_logfp);
    return 0;
}
(END)
```

Figure 5.2a: main() — socket creation, bind, listen

```c
/* ===== CLIENT THREAD ===== */
static void *client_thread(void *arg)
{
    session_t *s = (session_t *)arg;
    char line[BUF_SIZE];

    logmsg("CONNECTED %s:%d", s->peer, s->peer_port);

    while (1) {
        int r = read_line(s, line, sizeof(line));
        if (r <= 0) break;
        logmsg("CMD [%s:%d] %s", s->peer, s->peer_port, line);
        int is_quit = (strcasecmp(line, "QUIT") == 0);
        handle_line(s, line);
        if (is_quit) break;
    }

    if (s->mon_active) {
        pthread_mutex_lock(&s->mon_mutex);
        s->mon_stop = 1;
        pthread_mutex_unlock(&s->mon_mutex);
        pthread_join(s->mon_tid, NULL);
    }
    if (s->udp_fd) close(s->udp_fd);
    close(s->fd);
    logmsg("DISCONNECTED %s:%d", s->peer, s->peer_port);
    free(s);
    return NULL;
}
```

Figure 5.2b: main() accept loop and client_thread()

5.3 Line framing and byte counting

`read_line()` looks for `\n` in the session buffer, copies the line out, removes a trailing `\r` and shifts the remaining bytes forward with `memmove()`. If there is no complete line it calls `recv()` again. `recv_bytes()` reads exactly N raw bytes, first from the buffered data and then directly from the socket. Together they handle partial reads, multiple lines per read and exact-size file transfer.

```c
static int read_line(session_t *s, char *out, size_t outsz)
{
    while (1) {
        char *nl = memchr(s->rxbuf, '\n', s->rxlen);
        if (nl) {
            size_t linelen = nl - s->rxbuf;
            if (linelen >= outsz) linelen = outsz - 1;
            memcpy(out, s->rxbuf, linelen);
            out[linelen] = '\0';
            int rem = s->rxlen - (nl - s->rxbuf) - 1;
            memmove(s->rxbuf, nl + 1, rem);
            s->rxlen = rem;
            return (int)linelen;
        }
        if (s->rxlen >= BUF_SIZE - 1) { errno = EMSGSIZE; return -1; }
        int n = recv(s->fd, s->rxbuf + s->rxlen, BUF_SIZE - 1 - s->rxlen, 0)
;
        if (n <= 0) return n;
        s->rxlen += n;
    }
}

static int recv_bytes(session_t *s, void *dst, size_t n)
{
    char *p = dst;
    size_t got = 0;
    while (got < n && s->rxlen > 0) {
        size_t take = (n - got < (size_t)s->rxlen) ? (n - got) : (size_t)s->
rxlen;
        memcpy(p + got, s->rxbuf, take);
        memmove(s->rxbuf, s->rxbuf + take, s->rxlen - take);
        s->rxlen -= take;
        got += take;
    }
    while (got < n) {
        ssize_t r = recv(s->fd, p + got, n - got, 0);
        if (r <= 0) return (int)r;
        got += r;
    }
    return (int)got;
}

:
```

Figure 5.3: read_line() and recv_bytes()

5.4 Authentication and command dispatch

`handle_line()` splits the line with `strtok_r()` and compares the command name with `strcasecmp()`. AUTH is compared with `TOKEN`; on failure the Agent replies `ERR 001 AUTH_FAILED` and keeps the connection open. Before AUTH succeeds, every other command is answered with `ERR 003 NOT_AUTHENTICATED`. After authentication the command name selects the handler, and unknown commands receive `ERR 007 UNKNOWN_COMMAND`. Sizes and ports are validated, giving `ERR 008 BAD_REQUEST` for malformed values.



Figure 5.4: handle_line() authentication and dispatch

5.5 SYSINFO and LISTPROC

`get_sysinfo()` reads real values: the 1-minute load average from `/proc/loadavg`, used memory in MB as `MemTotal - MemAvailable` from `/proc/meminfo`, and uptime in seconds from `/proc/uptime`. `do_listproc()` runs the fixed command `ps -eo pid=,comm=` through `popen()` (first 40 processes), converts each line to `pid:name`, replaces spaces and commas inside names so the reply stays a single comma-separated line, and stops before the buffer would overflow.

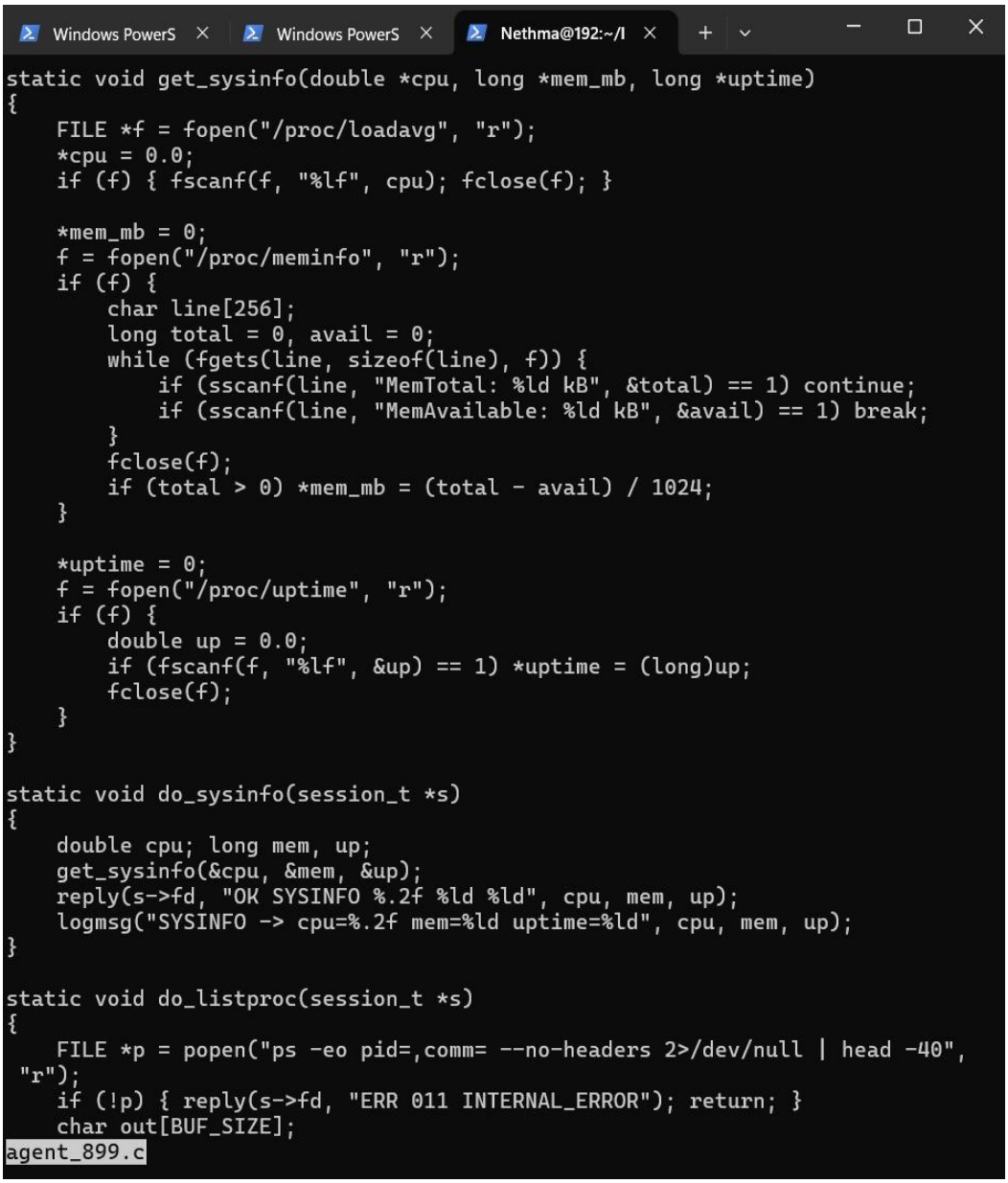


Figure 5.5: get_sysinfo() and do_listproc()

5.6 Restricted EXEC

The whitelist is a constant table mapping each allowed name (DATE, UPTIME, DISKFREE, HOSTNAME, WHOAMI) to a command string written in the source. The client's text is only compared with those names; it is never inserted into a shell command. Any other name returns `ERR 002 COMMAND_NOT_ALLOWED`. Output lines are joined with `|` so the response remains one line.

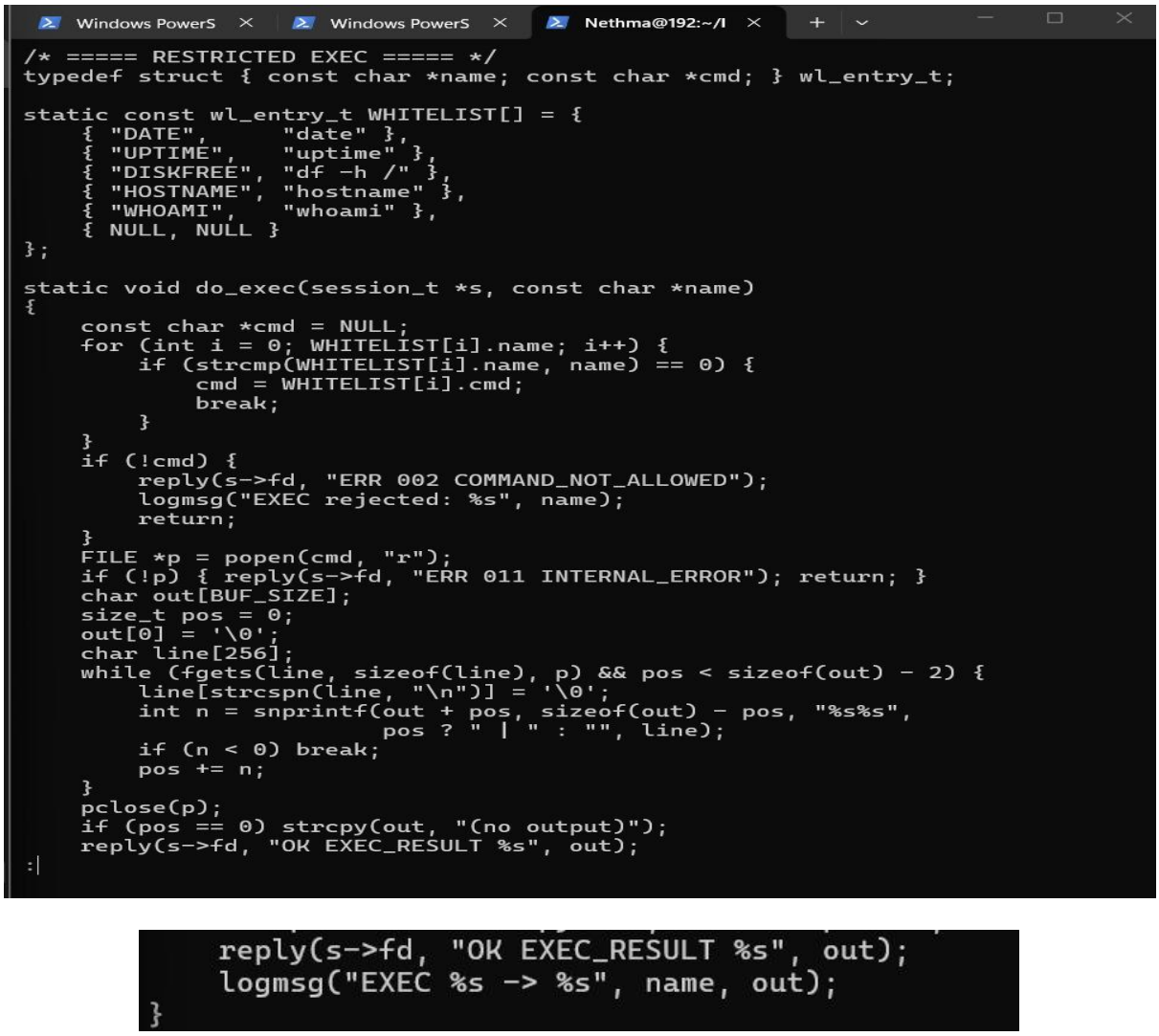


Figure 5.6: WHITELIST table and do_exec()

5.7 File upload and download

`valid_filename()` allows only letters, digits, dot, underscore and hyphen, and rejects names starting with a dot, which blocks path traversal such as `../x`. `do_put()` refuses files over 10 MB (`ERR 004`), streams exactly `<filesize>` bytes to `./agentfiles/IT24101899/<name>` and deletes the partial file if the client disconnects mid-transfer. `reject_put()` drains the announced bytes before replying with an error so framing is preserved. `do_get()` replies `OK FILE_SEND <name> <size>` and then sends exactly that many bytes, or `ERR 005 FILE_NOT_FOUND`.

```
/* ===== FILENAME VALIDATION ===== */
static int valid_filename(const char *n)
{
    if (!n || !*n || n[0] == '.') return 0;
    for (const char *p = n; *p; p++) {
        if (!( (*p >= 'a' && *p <= 'z') ||
               (*p >= 'A' && *p <= 'Z') ||
               (*p >= '0' && *p <= '9') ||
                *p == '.' || *p == '_' || *p == '-' ))
            return 0;
    }
    return 1;
}
```

Figure 5.7a: valid_filename()

```
/* ===== PUT / GET ===== */
static void reject_put(session_t *s, long size, int code, const char *reason
)
{
    char discard[BUF_SIZE];
    long rem = size;
    while (rem > 0) {
        size_t chunk = (rem < (long)sizeof(discard)) ? (size_t)rem : sizeof(
discard);
        int r = recv_bytes(s, discard, chunk);
        if (r <= 0) return;
        rem -= chunk;
    }
    reply(s->fd, "ERR %03d %s", code, reason);
}

static void do_put(session_t *s, const char *fname, long size)
{
    if (!valid_filename(fname)) {
        reject_put(s, size, 006, "INVALID_FILENAME");
        return;
    }
    if (size < 0 || size > MAX_UPLOAD_BYTES) {
        reject_put(s, size, 004, "FILE_TOO_LARGE");
        return;
    }
    char path[512];
    snprintf(path, sizeof(path), "%s/%s", STORE_DIR, fname);
    int fd = open(path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        reject_put(s, size, 011, "INTERNAL_ERROR");
        return;
    }
    char buf[BUF_SIZE];
    long rem = size;
    while (rem > 0) {
        size_t chunk = (rem < (long)sizeof(buf)) ? (size_t)rem : sizeof(buf)
;
        int r = recv_bytes(s, buf, chunk);
        if (r <= 0) {
            close(fd); unlink(path);
:|
```

```
            close(fd); unlink(path);
            logmsg("PUT aborted: %s", fname);
            return;
        }
        if (write(fd, buf, chunk) != (ssize_t)chunk) {
            close(fd); unlink(path);
            reply(s->fd, "ERR 011 INTERNAL_ERROR");
            return;
        }
        rem -= chunk;
    }
    close(fd);
    reply(s->fd, "OK FILE_RECEIVED %s", fname);
    logmsg("PUT %s (%ld bytes) from %s:%d", fname, size, s->peer, s->peer_po
rt);
}
```

Figure 5.7b: reject_put(), do_put()

```
static void do_get(session_t *s, const char *fname)
{
    if (!valid_filename(fname)) {
        reply(s->fd, "ERR 005 FILE_NOT_FOUND");
        return;
    }
    char path[512];
    snprintf(path, sizeof(path), "%s/%s", STORE_DIR, fname);
    struct stat st;
    if (stat(path, &st) < 0 || !S_ISREG(st.st_mode)) {
        reply(s->fd, "ERR 005 FILE_NOT_FOUND");
        logmsg("GET %s -> NOT FOUND", fname);
        return;
    }
    int fd = open(path, O_RDONLY);
    if (fd < 0) { reply(s->fd, "ERR 005 FILE_NOT_FOUND"); return; }

    char hdr[256];
    snprintf(hdr, sizeof(hdr), "OK FILE_SEND %s %ld SID:%s\n",
             fname, (long)st.st_size, SID_TAG);
    send(s->fd, hdr, strlen(hdr), MSG_NOSIGNAL);

    char buf[BUF_SIZE];
    ssize_t r;
    long total = 0;
:
```

```
    char buf[BUF_SIZE];
    ssize_t r;
    long total = 0;
    while ((r = read(fd, buf, sizeof(buf))) > 0) {
        ssize_t off = 0;
        while (off < r) {
            ssize_t w = send(s->fd, buf + off, r - off, MSG_NOSIGNAL);
            if (w <= 0) { close(fd); logmsg("GET send aborted"); return; }
            off += w;
        }
        total += r;
    }
    close(fd);
    logmsg("GET %s (%ld bytes) to %s:%d", fname, total, s->peer, s->peer_por
t);
}
:
```

Figure 5.7c: do_get()

5.8 UDP monitoring loop

`MONITOR START <port>` calls `start_monitor()`, which validates that no stream is active, creates a UDP socket, builds the Controller's address from the TCP peer IP and the requested port, and starts `monitor_thread()`. That thread sends `SYSINFO <cpu> <mem> <uptime> SID:9981` every 2 seconds. The sleep is split into 100 ms steps that check `mon_stop`, so `MONITOR STOP` (`stop_monitor()`), `QUIT` or a disconnect stop the stream almost immediately; the stopping code sets the flag under `mon_mutex` and joins the thread.

```
/* ===== UDP MONITOR ===== */
static void *monitor_thread(void *arg)
{
    session_t *s = (session_t *)arg;
    char buf[256];
    while (1) {
        pthread_mutex_lock(&s->mon_mutex);
        int stop = s->mon_stop;
        pthread_mutex_unlock(&s->mon_mutex);
        if (stop) break;

        double cpu; long mem, up;
        get_sysinfo(&cpu, &mem, &up);
        int n = snprintf(buf, sizeof(buf),
                         "SYSINFO <%.2f> <%ld> <%ld> SID:%s\n",
                         cpu, mem, up, SID_TAG);
        sendto(s->udp_fd, buf, n, 0,
               (struct sockaddr *)&s->ctrl_udp, sizeof(s->ctrl_udp));

        for (int i = 0; i < MON_INTERVAL_SEC * 10; i++) {
            usleep(100000);
            pthread_mutex_lock(&s->mon_mutex);
            stop = s->mon_stop;
            pthread_mutex_unlock(&s->mon_mutex);
            if (stop) return NULL;
        }
    }
    return NULL;
}
```

Figure 5.8: monitor_thread()

```
static void start_monitor(session_t *s, int port)
{
    if (s->mon_active) {
        reply(s->fd, "ERR 009 ALREADY_MONITORING");
        return;
    }
    if (!s->udp_fd) {
        s->udp_fd = socket(AF_INET, SOCK_DGRAM, 0);
        if (s->udp_fd < 0) { reply(s->fd, "ERR 011 INTERNAL_ERROR"); return;
 }
    }
    memset(&s->ctrl_udp, 0, sizeof(s->ctrl_udp));
    s->ctrl_udp.sin_family = AF_INET;
    s->ctrl_udp.sin_port   = htons(port);
    inet_pton(AF_INET, s->peer, &s->ctrl_udp.sin_addr);

    s->mon_stop = 0;
    s->mon_active = 1;
    if (pthread_create(&s->mon_tid, NULL, monitor_thread, s) != 0) {
        s->mon_active = 0;
        reply(s->fd, "ERR 011 INTERNAL_ERROR");
        return;
    }
    reply(s->fd, "OK MONITOR_STARTED");
    logmsg("MONITOR START udp_port=%d peer=%s", port, s->peer);
}

static void stop_monitor(session_t *s)
{
    if (!s->mon_active) {
        reply(s->fd, "ERR 010 NOT_MONITORING");
        return;
    }
    pthread_mutex_lock(&s->mon_mutex);
    s->mon_stop = 1;
    pthread_mutex_unlock(&s->mon_mutex);
    pthread_join(s->mon_tid, NULL);
    s->mon_active = 0;
    reply(s->fd, "OK MONITOR_STOPPED");
    logmsg("MONITOR STOP");
}

:|
```

Figure 5.8b: start_monitor() and stop_monitor()

5.9 Logging

`logmsg()` builds a timestamp with `localtime_r()`, locks `g_log_mutex`, appends the line to `remoteops_IT24101899.log` and flushes it. It is called for connections, every received command, authentication results, SYSINFO/LISTPROC/EXEC results, file transfers, monitor start/stop and disconnections, so all required events are recorded with timestamps.

```
/* ===== GLOBAL LOG ===== */
static FILE *g_logfp = NULL;
static pthread_mutex_t g_log_mutex = PTHREAD_MUTEX_INITIALIZER;

static void logmsg(const char *fmt, ...)
{
    va_list ap;
    time_t now = time(NULL);
    struct tm tm_info;
    char ts[32];
    localtime_r(&now, &tm_info);
    strftime(ts, sizeof(ts), "%Y-%m-%d %H:%M:%S", &tm_info);

    pthread_mutex_lock(&g_log_mutex);
    if (g_logfp) {
        fprintf(g_logfp, "[%s] ", ts);
        va_start(ap, fmt);
        vfprintf(g_logfp, fmt, ap);
        va_end(ap);
        fprintf(g_logfp, "\n");
        fflush(g_logfp);
    }
    pthread_mutex_unlock(&g_log_mutex);
}
```

Figure 5.9: logmsg()

5.10 Controller

The Controller connects to the Agent on port 9410 and reads commands from the user (`auth`, `sysinfo`, `listproc`, `exec`, `put <local> <remote>`, `get <remote> <local>`, `monitor start|stop`, `quit`). `print_reply()` prints each reply and warns if the SID tag is missing. `cmd_put()` sends `PUT <name> <size>` followed by the file bytes, and `cmd_get()` parses the `FILE_SEND` header and reads exactly the announced number of bytes; both print the throughput in bytes per second. `cmd_monitor()` binds a local UDP port (`start_udp_listener()`) before sending `MONITOR START`, and `udp_thread()` prints each datagram.

```
/* ===== LINE BUFFER ===== */
typedef struct { char buf[BUF_SIZE]; int len; } linebuf_t;

/* ===== UDP LISTENER ===== */
static int g_udp_fd = -1;
static volatile int g_udp_run = 1;

static void *udp_thread(void *arg)
{
    (void)arg;
    char buf[BUF_SIZE];
    while (g_udp_run) {
        struct sockaddr_in from;
        socklen_t fl = sizeof(from);
        int n = recvfrom(g_udp_fd, buf, sizeof(buf) - 1, 0,
                         (struct sockaddr *)&from, &fl);
        if (n <= 0) { if (!g_udp_run) break; continue; }
        buf[n] = '\0';
        time_t now = time(NULL);
        char ts[16];
        strftime(ts, sizeof(ts), "%H:%M:%S", localtime(&now));
        printf("[UDP %s] %s", ts, buf);
        fflush(stdout);
    }
    return NULL;
}
```

Figure 5.10a: udp_thread() (controller_899.c)

```
Windows PowerS   Windows PowerS   Nethma@192:~/l
/* ===== PUT ===== */
static void cmd_put(int fd, linebuf_t *lb, const char *local, const char *re
mote)
{
    struct stat st;
    if (stat(local, &st) < 0 || !S_ISREG(st.st_mode)) {
        fprintf(stderr, "PUT: cannot stat %s\n", local); return;
    }

    send_line(fd, "PUT %s %ld", remote, (long)st.st_size);

    int lfd = open(local, O_RDONLY);
    if (lfd < 0) { perror("open"); return; }

    char buf[BUF_SIZE];
    long rem = st.st_size;
    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);

    while (rem > 0) {
        ssize_t r = read(lfd, buf, sizeof(buf));
        if (r <= 0) break;
        ssize_t off = 0;
        while (off < r) {
            ssize_t w = send(fd, buf + off, r - off, MSG_NOSIGNAL);
            if (w <= 0) { close(lfd); return; }
            off += w;
        }
        rem -= r;
    }
    close(lfd);

    clock_gettime(CLOCK_MONOTONIC, &t1);
    double secs = (t1.tv_sec - t0.tv_sec) +
                  (t1.tv_nsec - t0.tv_nsec) / 1e9;
    if (secs <= 0) secs = 0.001;
    printf("[throughput] PUT %.0f bytes/s\n", st.st_size / secs);

    char line[BUF_SIZE];
    int n = read_line(fd, lb, line, sizeof(line));
    if (n > 0) print_reply(line);
}
:|
```

Figure 5.10b: cmd_put()(controller_899.c)



Figure 5.10c: cmd_get() (controller_899.c)

6. Execution and Output Screenshots

6.1 Agent starting up

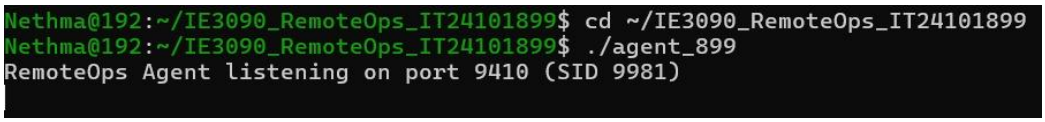


Figure 6.1: Agent starting up

6.2 Successful AUTH

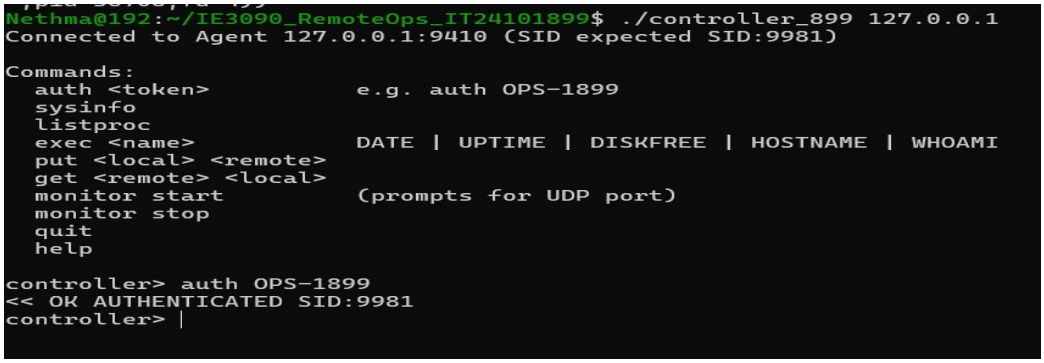


Figure 6.2: Successful AUTH

6.3 SYSINFO, LISTPROC and EXEC

```
Commands:
  auth <token>          e.g. auth OPS-1899
  sysinfo
  listproc
  exec <name>           DATE | UPTIME | DISKFREE | HOSTNAME | WHOAMI
  put <local> <remote>
  get <remote> <local>
  monitor start         (prompts for UDP port)
  monitor stop
  quit
  help

controller> auth OPS-1899
<< OK AUTHENTICATED SID:9981
controller> sysinfo
<< OK SYSINFO 0.02 1381 6266 SID:9981
controller> listproc
<< OK PROCs ______1_systemd,______2_kthreadd,______3_pool_workqueue_release,______4_kworker/R-rcu_gp,______5_kworker/R-sync_wq,______6_kworker/R-kvfree_rcu_
reclaim,______7_kworker/R-slub_flushwq,______8_kworker/R-netns,______9_kworker/0:0-ata_sff,_____12_kworker/u16:0-events_unbound,_____13_kworker/u16:1-ipv6_a
ddrconf,_____14_kworker/R-mm_percpu_wq,_____15_ksoftirqd/0,_____16_rcu_preempt,_____17_rcu_exp_par_gp_kthread_worker/0,_____18_rcu_exp_gp_kthread_worker,___
__19_migration/0,_____20_idle_inject/0,_____21_cpuhp/0,_____22_cpuhp/1,_____23_idle_inject/1,_____24_migration/1,_____25_ksoftirqd/1,_____28_cpuhp/2,_____29
_idle_inject/2,_____30_migration/2,_____31_ksoftirqd/2,_____33_kworker/2:0H-events_highpri,_____34_cpuhp/3,_____35_idle_inject/3,_____36_migration/3,_____37
_ksoftirqd/3,_____40_kworker/u17:0-events_unbound,_____45_kworker/u19:1-events_unbound,_____47_kdevtmpfs,_____48_kworker/R-inet_frag_wq,_____49_rcu_tasks_kt
hread,_____50_rcu_tasks_rude_kthread,_____51_rcu_tasks_trace_kthread,_____52_kworker/u20:1-flush-253:0 SID:9981
controller> exec DATE
<< OK EXEC_RESULT Sun Oct  4 12:56:21 PM EDT 2026 SID:9981
controller> exec HOSTNAME
<< OK EXEC_RESULT 192.168.1.6 SID:9981
controller>
```

Figure 6.3: SYSINFO, LISTPROC and EXEC

6.4 Successful PUT and GET

```
controller> put testfile.txt testfile.txt
[throughput] PUT 320688 bytes/s
<< OK FILE_RECEIVED testfile.txt SID:9981
controller>
```

```
controller> get testfile.txt downloaded.txt
<< OK FILE_SEND testfile.txt 37 SID:9981
[throughput] GET 765713 bytes/s
Saved downloaded.txt (37 bytes)
controller>
```

Figure 6.4: Successful PUT and GET

6.5 UDP monitoring stream

```
Nethma@192:~/IE3090_RemoteOps_IT24101899$ ./controller_899 127.0.0.1
Connected to Agent 127.0.0.1:9410 (SID expected SID:9981)

Commands:
  auth <token>          e.g. auth OPS-1899
  sysinfo
  listproc
  exec <name>           DATE | UPTIME | DISKFREE | HOSTNAME | WHOAMI
  put <local> <remote>
  get <remote> <local>
  monitor start         (prompts for UDP port)
  monitor stop
  quit
  help

controller> auth OPS-1899
<< OK AUTHENTICATED SID:9981
controller> monitor start
UDP listen port (e.g. 9500): 9500
<< OK MONITOR_STARTED SID:9981
controller> [UDP 13:43:31] SYSINFO <0.11> <1383> <9109> SID:9981
[UDP 13:43:33] SYSINFO <0.11> <1383> <9111> SID:9981
[UDP 13:43:35] SYSINFO <0.10> <1383> <9113> SID:9981
[UDP 13:43:37] SYSINFO <0.10> <1383> <9115> SID:9981
[UDP 13:43:39] SYSINFO <0.10> <1384> <9117> SID:9981
[UDP 13:43:42] SYSINFO <0.10> <1384> <9119> SID:9981
[UDP 13:43:44] SYSINFO <0.10> <1384> <9122> SID:9981
[UDP 13:43:46] SYSINFO <0.09> <1384> <9124> SID:9981
[UDP 13:43:48] SYSINFO <0.09> <1384> <9126> SID:9981
[UDP 13:43:50] SYSINFO <0.08> <1384> <9128> SID:9981
[UDP 13:43:52] SYSINFO <0.08> <1384> <9130> SID:9981
[UDP 13:43:54] SYSINFO <0.08> <1384> <9132> SID:9981
[UDP 13:43:56] SYSINFO <0.07> <1384> <9134> SID:9981
[UDP 13:43:58] SYSINFO <0.07> <1384> <9136> SID:9981
[UDP 13:44:00] SYSINFO <0.07> <1384> <9138> SID:9981
[UDP 13:44:02] SYSINFO <0.07> <1384> <9140> SID:9981
[UDP 13:44:04] SYSINFO <0.07> <1384> <9142> SID:9981
[UDP 13:44:06] SYSINFO <0.06> <1384> <9144> SID:9981
[UDP 13:44:08] SYSINFO <0.06> <1384> <9146> SID:9981
[UDP 13:44:10] SYSINFO <0.06> <1384> <9148> SID:9981
[UDP 13:44:12] SYSINFO <0.06> <1384> <9150> SID:9981
[UDP 13:44:14] SYSINFO <0.06> <1384> <9152> SID:9981
[UDP 13:44:16] SYSINFO <0.05> <1384> <9154> SID:9981
```

Figure 6.5: UDP monitoring stream

6.6 Error cases handled gracefully

```
Nethma@192:~/IE3090_RemoteOps_IT24101899$ ./controller_899 127.0.0.1
Connected to Agent 127.0.0.1:9410 (SID expected SID:9981)

Commands:
  auth <token>          e.g. auth OPS-1899
  sysinfo
  listproc
  exec <name>           DATE | UPTIME | DISKFREE | HOSTNAME | WHOAMI
  put <local> <remote>
  get <remote> <local>
  monitor start         (prompts for UDP port)
  monitor stop
  quit
  help

controller> sysinfo
<< ERR 003 NOT_AUTHENTICATED SID:9981
controller> auth WRONGTOKEN
<< ERR 001 AUTH_FAILED SID:9981
controller> auth OPS-1899
ex<< OK AUTHENTICATED SID:9981
controller> auth OPS-1899
unknown command: > (try 'help')
controller> auth OPS-1899
<< ERR 008 BAD_REQUEST SID:9981
controller> exec rm -rf /
<< ERR 002 COMMAND_NOT_ALLOWED SID:9981
controller>
```

Figure 6.6: Error cases handled gracefully

6.7 Five simultaneous Controllers

```
Nethma@192:~/IE3090_RemoteOps_IT24101899$ cd ~/IE3090_RemoteOps_IT24101899
Nethma@192:~/IE3090_RemoteOps_IT24101899$ for i in 1 2 3 4 5; do
    (echo "auth OPS-1899"; sleep 1; echo "sysinfo"; sleep 1; echo "quit") |
./controller_899 127.0.0.1 > /tmp/ctrl_$i.log 2>&1 &
done
wait
[1] 36080
[2] 36083
[3] 36086
[4] 36089
[5] 36091
[1]   Done                    ( echo "auth OPS-1899"; sleep 1; echo "sysinfo
"; sleep 1; echo "quit" ) | ./controller_899 127.0.0.1 > /tmp/ctrl_$i.log 2>
&1
[2]   Done                    ( echo "auth OPS-1899"; sleep 1; echo "sysinfo
"; sleep 1; echo "quit" ) | ./controller_899 127.0.0.1 > /tmp/ctrl_$i.log 2>
&1
[3]   Done                    ( echo "auth OPS-1899"; sleep 1; echo "sysinfo
"; sleep 1; echo "quit" ) | ./controller_899 127.0.0.1 > /tmp/ctrl_$i.log 2>
&1
[5]+  Done                    ( echo "auth OPS-1899"; sleep 1; echo "sysinfo
"; sleep 1; echo "quit" ) | ./controller_899 127.0.0.1 > /tmp/ctrl_$i.log 2>
&1
Nethma@192:~/IE3090_RemoteOps_IT24101899$ grep CONNECTED remoteops_IT2410189
9.log | tail -10
[2026-10-04 13:52:31] CONNECTED 127.0.0.1:39482
[2026-10-04 13:52:31] CONNECTED 127.0.0.1:39492
[2026-10-04 13:52:31] CONNECTED 127.0.0.1:39496
[2026-10-04 13:52:31] CONNECTED 127.0.0.1:39510
[2026-10-04 13:52:31] CONNECTED 127.0.0.1:39514
[2026-10-04 13:52:33] DISCONNECTED 127.0.0.1:39492
[2026-10-04 13:52:33] DISCONNECTED 127.0.0.1:39496
[2026-10-04 13:52:33] DISCONNECTED 127.0.0.1:39510
[2026-10-04 13:52:33] DISCONNECTED 127.0.0.1:39514
[2026-10-04 13:52:33] DISCONNECTED 127.0.0.1:39482
Nethma@192:~/IE3090_RemoteOps_IT24101899$
```

Figure 6.7: Five simultaneous Controllers

7. Testing Summary

Each test covers a mandatory feature from section 2.2 of the brief. The commands are typed in the Controller (`controller_899`) unless stated otherwise. Write the real outcome of each test in the last column.

#	Test (what I did)	Expected result	Actual result
1	`sysinfo` before logging in	`ERR 003 NOT_AUTHENTICATED SID:9981`	Passed — got ERR 003 NOT_AUTHENTICATED SID:9981
2	`auth wrongtoken`	`ERR 001 AUTH_FAILED SID:9981`; connection stays open	Passed — got ERR 001 AUTH_FAILED SID:9981, connection stayed open
3	`auth OPS-1899`	`OK AUTHENTICATED SID:9981`	Passed — got OK AUTHENTICATED SID:9981
4	`sysinfo`	`OK SYSINFO <cpu> <mem_mb> <uptime> SID:9981`	Passed — OK SYSINFO 0.82 1381 6266 SID:9981
5	`listproc`	`OK PROCS pid:name,... SID:9981`	Passed — OK PROCs ... SID:9981
6	`exec DATE`, `UPTIME`, `DISKFREE`, `HOSTNAME`, `WHOAMI`	`OK EXEC_RESULT <output> SID:9981` for each	Passed — five OK EXEC_RESULT replies
7	`exec rm`	`ERR 002 COMMAND_NOT_ALLOWED SID:9981`	Passed — ERR 002 COMMAND_NOT_ALLOWED SID:9981
8	`put` a small text file and a 3 MB random file	`OK FILE_RECEIVED <name> SID:9981`; both files appear in `./agentfiles/IT24101899/`	Passed — both files in ./agentfiles/IT24101899/
9	`get` both files, then compare with `cmp`	Downloaded files byte-for-byte identical (no `cmp` output)	Passed — cmp reported no differences

#	Test (what I did)	Expected result	Actual result
10	`get nothere.txt <local>`	`ERR 005 FILE_NOT_FOUND SID:9981`	Passed — ERR 005 FILE_NOT_FOUND SID:9981
11	`put` an 11 MB file, then `sysinfo` on the same connection	`ERR 004 FILE_TOO_LARGE SID:9981`; next command answered correctly (framing intact)	Passed — ERR 004 FILE_TOO_LARGE, framing intact
12	`put small.txt ../evil`	`ERR 006 INVALID_FILENAME SID:9981`; nothing written outside the storage folder	Passed — ERR 006 INVALID_FILENAME, nothing written outside
13	`monitor start` (port 9500), wait about 6 s, `monitor stop`	`OK MONITOR_STARTED`; about one datagram per 2 s ending in `SID:9981`; `OK MONITOR_STOPPED`; datagrams stop	Passed — datagrams every 2 s, then stopped cleanly
14	Send `FOO` using bash `/dev/tcp` (the Controller refuses unknown commands locally)	`ERR 007 UNKNOWN_COMMAND SID:9981`	Passed — ERR 007 UNKNOWN_COMMAND SID:9981
15	Send `AUTH OPS-`, pause 1 s, then `1899\nSYSINFO\nEXECDATE\n` using bash `/dev/tcp`	Three correct replies: partial line reassembled and two commands in one buffer both handled	Passed — partial line reassembled, both commands handled
16	Start 6 Controllers at once, each does `auth` + `sysinfo`	All 6 receive correct replies	Passed — all 6 received correct replies
17	Open a raw connection, start a PUT, close it mid-transfer	Agent does not crash; partial file removed; `PUT aborted` and `DISCONNECTED` in the log; new clients still served	Passed — Agent did not crash; PUT aborted; log shows DISCONNECTED

#	Test (what I did)	Expected result	Actual result
18	Inspect `remoteops_IT24101899.log`	Timestamped CONNECTED, CMD, results, PUT/GET, MONITOR and DISCONNECTED lines	Passed — all events timestamped in the log

8. Design Rationale and Assumptions

8.1 Key design decisions

- **Single reply() function:** adding the SID tag in one place guarantees every response is compliant and makes the format easy to change.
- **Buffered reader shared by lines and file data:** one buffer per session avoids losing bytes that arrive in the same `recv()` as a command, which is the usual source of PUT bugs.
- **Whitelist as a constant table:** user input is only compared with fixed names and never reaches a shell, which satisfies the security requirement that EXEC must not run arbitrary commands.
- **Filename validation:** because PUT filenames come from the network, only a safe character set is accepted, which prevents writing outside `./agentfiles/IT24101899/`.
- **Draining rejected uploads:** the protocol sends file bytes immediately after the PUT line, so the Agent must consume them even when refusing the file; otherwise they would be parsed as commands.
- **Monitor thread per session with a stop flag and join:** guarantees the UDP stream stops on MONITOR STOP, QUIT or an abrupt disconnect.

8.2 Assumptions where the brief leaves a choice

- UDP monitoring interval is **2 seconds**; the maximum upload size is **10 MB**.
- A failed AUTH does not close the connection; the Controller may retry. Commands stay locked until AUTH succeeds, and a second AUTH on an authenticated session returns `ERR 008 BAD_REQUEST`.
- Numeric error codes chosen: 001 AUTH_FAILED, 002 COMMAND_NOT_ALLOWED, 003 NOT_AUTHENTICATED, 004 FILE_TOO_LARGE, 005 FILE_NOT_FOUND, 006 INVALID_FILENAME, 007 UNKNOWN_COMMAND, 008 BAD_REQUEST, 009 ALREADY_MONITORING, 010 NOT_MONITORING, 011 INTERNAL_ERROR.
- `MONITOR STOP` when no stream is running returns `ERR 010`; starting a second stream on the same session returns `ERR 009`; a port outside 1–65535 returns `ERR 008`.
- Memory used is `MemTotal - MemAvailable` in megabytes; CPU load is the 1-minute load average; LISTPROC returns the first 40 processes as `pid:name`; multi-line EXEC output is joined with `|` .
- Command names are case-insensitive on the Agent; EXEC names (such as `DATE`) are case-sensitive and must match the whitelist exactly.
- Uploading a file with an existing name overwrites it. Files are stored in the registration-number folder as required.
- UDP datagrams go to the IP address of the Controller's TCP connection on the port given in `MONITOR START` (the Controller asks for it and binds it first). IPv4 only.

9. Optional Extension and Limitations

Implemented extension: the Controller measures and prints transfer throughput in bytes per second for every PUT and GET (section 2.6 of the brief).

Known limitations:

- The control channel is plain TCP (no TLS), so the token and file contents travel unencrypted.
- There is no limit on authentication attempts and no cap on the number of concurrent threads.
- UDP is unreliable by nature, so monitoring datagrams can be lost or reordered; this is acceptable for periodic statistics.
- LISTPROC is limited to the first 40 processes. The Agent targets Linux (`/proc`, `ps`) and IPv4 only.

10. Build and Run Instructions

```
make -f Makefile_899 clean
make -f Makefile_899
./agent_899
./controller_899 127.0.0.1
controller> auth OPS-1899
<< OK AUTHENTICATED SID:9981
controller> sysinfo
<< OK SYSINFO 0.82 1381 6266 SID:9981
...
```